

Número: _____ Nome: _____

1. Correção (5 valores)

Considere a seguinte função que testa se uma string é palíndromo.

Escreva um invariante e um variante que permitam provar a **correção total** desta função.

```
int palindrome (char s[], int N) {  
    // Pre: N>=0  
    int r, i=0, j=N-1;  
    while (i<j && s[i] == s[j]){  
        i++;j--;  
    }  
    r = i==j;  
    // Pos: r == \forall_{0<=k<N} s[k] == s[N-k-1]  
    return r ;  
}
```

2. Complexidade de algoritmos recursivos (5 valores)

Considere a seguinte definição alternativa da função palindrome.

Escreva e resolva uma relação de recorrência que traduza o custo desta função (número de caracteres de s consultados) no pior caso.

```
int palindrome (char s[], int N) {  
    if (N<2) return 1;  
    if (s[0] != s[N-1]) return 0;  
    return palindrome (s+1,N-2);  
}
```

Número: _____ Nome: _____

3. Complexidade de algoritmos iterativos (5 valores)

Relembre a função `int inc (int b[], int N)` que incrementa um número representado por um array `b` de `N` bits. Em termos do número de bits trocados, a função tem um pior caso linear (no caso em que todos os $(N-1)$ bits menos significativos são 1).

Considere agora uma variante desta função – `inc2` – que recebe também um array `v` (com a mesma dimensão de `b`), e em que, sempre que se troca o bit na posição `i`, invoca uma função `ins(v,i)` que processa o array `v[0..i]` e tem um custo linear: $T_{ins}(N) = N$

```
int inc2 (int b[], int v[], int N){
    int r=0;
    for (i=0; i<N && b[i] == 1; i++){
        ins (v,i);
        b[i] = 0;
    }
    if (i==N) r = 1;
    else { b[i] = 1; ins (v, i);}
    return r;
}
```

Calcule o custo de `inc2` no pior caso (que corresponde ao pior caso da função `inc` original).

4. Análise de caso médio (5 valores)

Apresente uma expressão (eventualmente envolvendo somatórios) que traduza o custo médio da função `inc2` definida na questão anterior.